MacDependency

Modern Binary Dependency Analysis for macOS and Apple Silicon

An Open-Source Development and Binary Inspection Tool

Project repository
https://github.com/sedwards/macdependency

Platform: macOS | Architectures: ARM64 and x86_64
License: GNU General Public License v3.0

Contents

1. Executive Summary

2. 1. The Problem

3. 2. Project Background

4. 3. Core Capabilities

5. 4. Benefits to the macOS Development Community

6. 5. Practical Applications

7. 6. Comparison with Existing Tools

8. 7. Technical Considerations and Current Limitations

9. 8. Future Development Opportunities

10. 9. Open-Source Philosophy

11. 10. Conclusion

Executive Summary

Software development on macOS increasingly involves complex relationships between executable binaries, dynamic libraries, frameworks, and external dependencies. Understanding these relationships is essential when developing, debugging, maintaining, and distributing software.

For many years, Windows developers have benefited from tools such as Microsoft's Dependency Walker, which provides a visual representation of DLL dependencies, imported and exported functions, and symbol information.

Historically, macOS developers have relied primarily on command-line utilities such as Apple's otool, nm, and other Mach-O inspection tools. Although powerful, these utilities require developers to understand command-line output and manually correlate information across multiple binaries.

MacDependency addresses this gap. It is an open-source macOS application designed to provide graphical binary dependency analysis, allowing developers to examine executable files, dynamic libraries, and frameworks through an organized visual interface.

Originally developed by another open-source contributor, the project had become inactive. It has since been forked and brought forward with renewed development, including modernization and support for Apple's ARM-based processors.

The objective is to provide macOS developers with a practical, accessible, and extensible alternative to traditional command-line binary inspection tools. By combining dependency visualization, architecture awareness, symbol inspection, and command-line integration, MacDependency offers a valuable development tool for the modern macOS ecosystem.

1. The Problem

Modern applications rarely operate independently. A typical macOS application depends on numerous system frameworks, third-party libraries, dynamically linked components, and other executable resources.

When an application fails to launch, encounters linking errors, or behaves unexpectedly, identifying the underlying dependency problem can be challenging.

Common development problems include missing or incorrectly referenced dynamic libraries; unresolved or missing symbols; incompatible binary architectures; unexpected library dependencies; framework version incompatibilities; incorrect linking configurations; differences between development and deployment environments; and understanding the internal structure of third-party binaries.

Developers frequently need to determine precisely which libraries an application requires and which functions those libraries provide. Although macOS includes several command-line utilities for inspecting binaries, these tools do not provide the same consolidated graphical experience traditionally available to Windows developers.

MacDependency is intended to make this information more accessible.

2. Project Background

MacDependency originated as an open-source macOS project intended to provide functionality comparable to Windows Dependency Walker. The original project provided graphical inspection of Mach-O binaries and their dependent libraries and frameworks.

Following a period of inactivity, the project was forked and continued with the objective of making it useful on current macOS systems.

An important motivation for this work is Apple's transition from Intel processors to Apple Silicon. The introduction of ARM64-based Macs created additional requirements for development utilities, particularly those used to inspect

executable formats, architectures, and linking relationships.

The continued development of MacDependency addresses the need for a modern binary inspection utility that operates within the contemporary Apple development environment. The project retains its open-source foundation and provides a platform for additional improvements and community contributions.

3. Core Capabilities

3.1 Dependency Tree Visualization

One of MacDependency's primary capabilities is its hierarchical presentation of binary dependencies. Rather than requiring developers to interpret lengthy command-line output, the application presents dependency relationships in a graphical tree.

This allows developers to identify libraries required by an executable, follow dependencies between libraries, understand an application's linking relationships, investigate unexpected dependencies, and identify potentially missing components.

Visual dependency analysis is particularly useful when working with large applications containing numerous frameworks and dynamically linked libraries.

3.2 Symbol Inspection

Understanding the relationship between libraries requires more than knowing which files depend on one another. Developers frequently need to inspect the symbols contained within individual binaries.

MacDependency provides symbol-table inspection, allowing developers to examine exported functions and symbol information. This is useful when investigating available library interfaces, exported functions, symbol naming conventions, linking relationships, missing or unresolved symbols, and compiler-generated symbol names.

Where supported, symbol demangling makes otherwise difficult-to-read compiler-generated names more understandable. This is especially valuable when investigating C++ applications and libraries.

3.3 Multiple Architecture Support

Apple's transition to Apple Silicon introduced a fundamental architectural change to the Mac platform. Modern macOS development must account for both Intel x86_64 and ARM64 binaries.

MacDependency provides architecture-aware inspection, allowing developers to examine binaries targeting different processor architectures. This is important when porting applications from Intel to Apple Silicon, maintaining cross-platform software, investigating architecture-specific linking problems, developing universal applications, examining third-party libraries, and diagnosing incompatible binary components.

Native Apple Silicon support makes MacDependency particularly relevant to developers using modern M-series Macs.

3.4 Command-Line Integration

Although graphical inspection is a central feature, MacDependency also supports command-line operation. Developers can launch the application with executable files, applications, or binary paths as arguments.

Multiple files can be supplied, allowing separate inspection windows to be opened for different binaries. This provides an opportunity to integrate MacDependency into existing development workflows. Developers can launch an inspection directly from a terminal session after compiling an application or library.

3.5 Application and Framework Inspection

MacDependency can inspect supported Mach-O executables, dynamic libraries, and frameworks. This allows developers to examine both their own compiled applications and third-party software components.

The ability to inspect different binary types makes the application useful across application development, library development, software maintenance, and binary compatibility investigations.

4. Benefits to the macOS Development Community

4.1 A Graphical Alternative to Command-Line Inspection

macOS provides powerful binary inspection utilities, including otool, nm, lipo, and dyld-related utilities. However, developers often need to combine multiple commands to understand the complete dependency structure of an application.

MacDependency brings important aspects of this information together in a graphical environment. This reduces the need to manually correlate command output and makes binary analysis more accessible.

4.2 Improved Debugging and Troubleshooting

Dependency-related problems can consume significant development time. MacDependency helps developers investigate the underlying structure of binaries and identify relationships that may contribute to runtime or linking problems.

This can be particularly helpful when working with third-party libraries, complex application builds, legacy software, cross-architecture applications, and software being ported to Apple Silicon.

4.3 Supporting Apple Silicon Development

Apple's M-series processors represent a major change in the macOS computing environment. Development tools must evolve alongside the hardware and operating system.

MacDependency contributes to this transition by providing a binary inspection utility that supports ARM64 alongside Intel architecture analysis. This is valuable for developers maintaining software originally written for Intel Macs while transitioning to native Apple Silicon builds.

4.4 Open-Source Accessibility

MacDependency is distributed under the GNU General Public License version 3. Its open-source availability provides several advantages: developers can inspect the source code; organizations can evaluate and modify the application; contributors can submit improvements; additional functionality can be developed; and the project can evolve through community participation.

Unlike proprietary binary analysis utilities, MacDependency provides an opportunity for developers to understand and extend the underlying implementation.

5. Practical Applications

Application Development. Developers can examine their own compiled applications to understand which libraries and frameworks they depend upon.

Library Development. Library developers can inspect exported symbols and understand how their compiled components expose functionality to other applications.

Software Porting. When migrating applications from Intel to Apple Silicon, developers can investigate architecture-specific dependencies and linking relationships.

Debugging. When applications encounter missing-library or symbol-resolution problems, MacDependency provides additional visibility into the underlying binary structure.

Software Maintenance. Developers maintaining older applications can investigate existing dependencies and better understand the software's internal organization.

Binary Analysis. Researchers and developers can examine Mach-O binary structures and investigate the relationships between compiled software components.

6. Comparison with Existing Tools

MacDependency occupies an important position between traditional command-line utilities and more specialized binary analysis applications. It is not intended to replace every specialized binary analysis utility; instead, it provides a focused graphical tool for understanding macOS binary dependencies.

Capability	macOS CLI tools	MacDependency	Windows Dependency Walker
Dependency inspection	Yes	Yes	Yes
Graphical dependency tree	No	Yes	Yes
Mach-O inspection	Yes	Yes	No
PE/DLL inspection	With other tools	No	Yes
Architecture inspection	Yes	Yes	Yes
Symbol inspection	Yes	Yes	Yes
Exported function inspection	Yes	Yes	Yes
Command-line integration	Yes	Yes	Yes
Native macOS interface	No	Yes	No

7. Technical Considerations and Current Limitations

MacDependency operates within Apple's Mach-O binary environment.

One important consideration is Apple's introduction of the dyld shared cache in macOS 11 Big Sur. Modern macOS system libraries and frameworks are frequently stored within this shared cache rather than existing as individually accessible files in their traditional filesystem locations.

Consequently, directly analyzing certain system frameworks and libraries may require extracting the relevant binaries from the shared cache first. This is an operating-system architectural consideration rather than a fundamental limitation of dependency analysis.

Third-party applications, independently compiled binaries, and accessible dynamic libraries remain important targets for analysis.

8. Future Development Opportunities

MacDependency provides a foundation for continued development of macOS binary analysis capabilities. Potential areas for future improvement include expanded symbol import and export analysis; more comprehensive C++ symbol demangling; enhanced dependency visualization; improved universal binary inspection; additional ARM64 and ARM64e analysis capabilities; better dyld shared-cache integration; dependency comparison between binary versions; improved detection and reporting of unresolved dependencies; automated binary compatibility analysis; and additional command-line automation capabilities.

These opportunities provide a roadmap for evolving MacDependency into a more comprehensive macOS development and binary inspection environment.

9. Open-Source Philosophy

MacDependency is built upon the principle that useful development tools should remain accessible to the broader software development community.

The project builds upon existing open-source work rather than attempting to recreate established functionality behind proprietary licensing. By continuing development of an otherwise inactive project, MacDependency helps preserve existing software development knowledge while extending its usefulness to modern Apple hardware.

The GPL-3.0 licensing model allows the project to remain open to inspection, modification, and redistribution under the terms of that license. Community participation and contributions are encouraged.

10. Conclusion

As macOS continues to evolve, developers need tools that keep pace with changes in hardware architectures, binary formats, and operating-system infrastructure. Understanding binary dependencies remains a fundamental part of software development, debugging, and maintenance.

MacDependency provides a graphical approach to this problem, bringing dependency visualization and symbol inspection into a native macOS application.

By continuing development of an established open-source project and supporting Apple's modern ARM-based hardware, MacDependency helps address an important gap in the macOS developer toolkit.

The project's objective is straightforward: provide macOS developers with an accessible, open-source, and modern alternative for inspecting binary dependencies, understanding symbol relationships, and investigating the internal structure of compiled software.

OPEN SOURCE. NATIVE macOS. BUILT FOR MODERN APPLE SILICON.

https://github.com/sedwards/macdependency